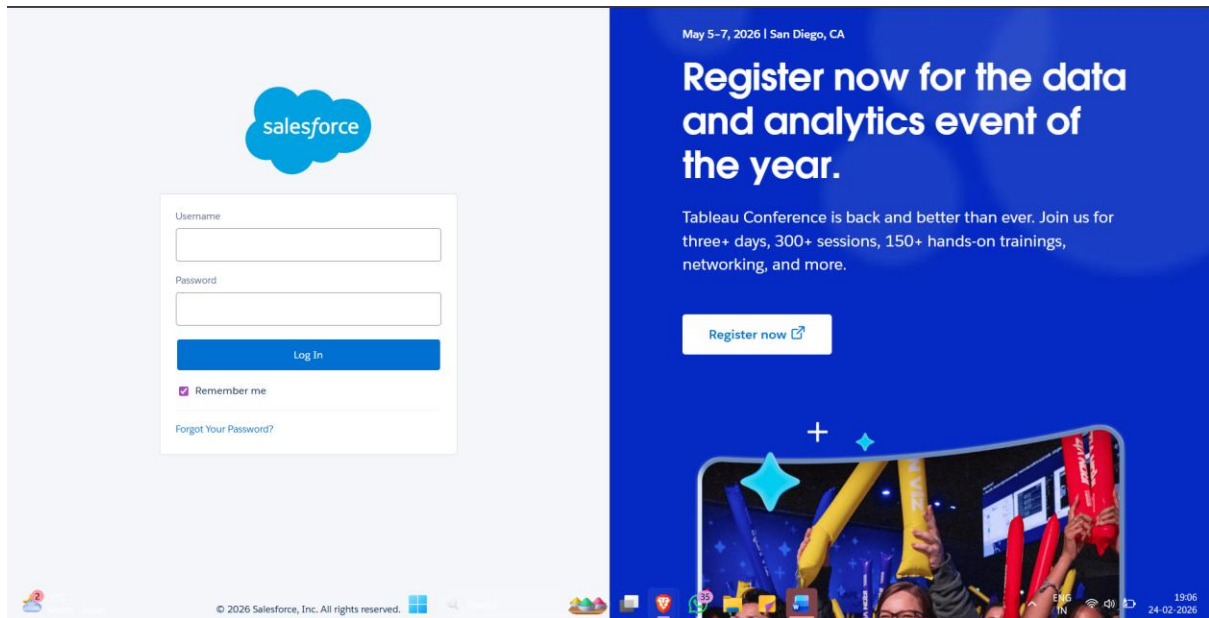
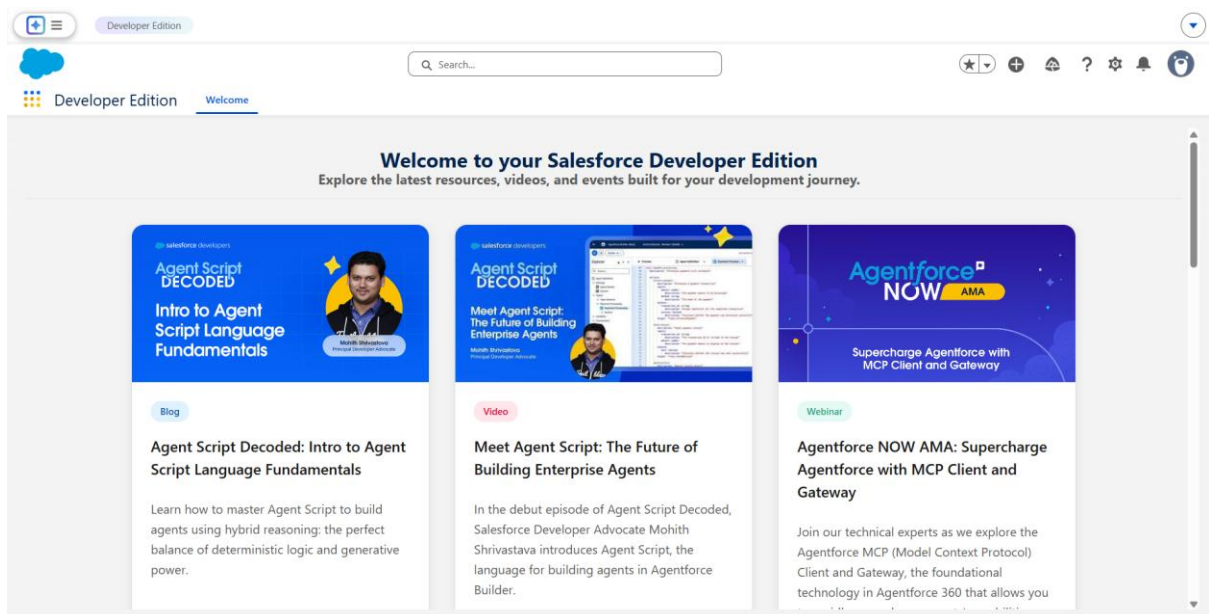


1. Account Access

- **Sign up & Login:** Go to developer.salesforce.com/signup to create your free Developer Edition. Once your email is verified, log in at login.salesforce.com.

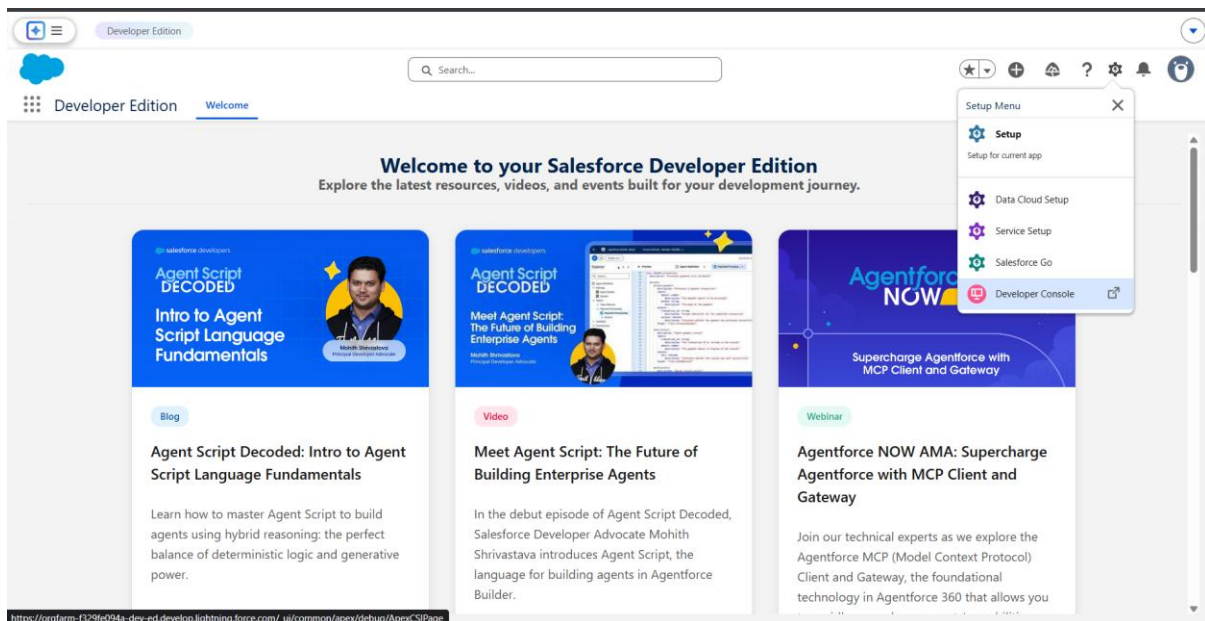


- **Enter Setup:** Once logged in, you will be on the Setup Home page.



2. Opening the Developer Tools

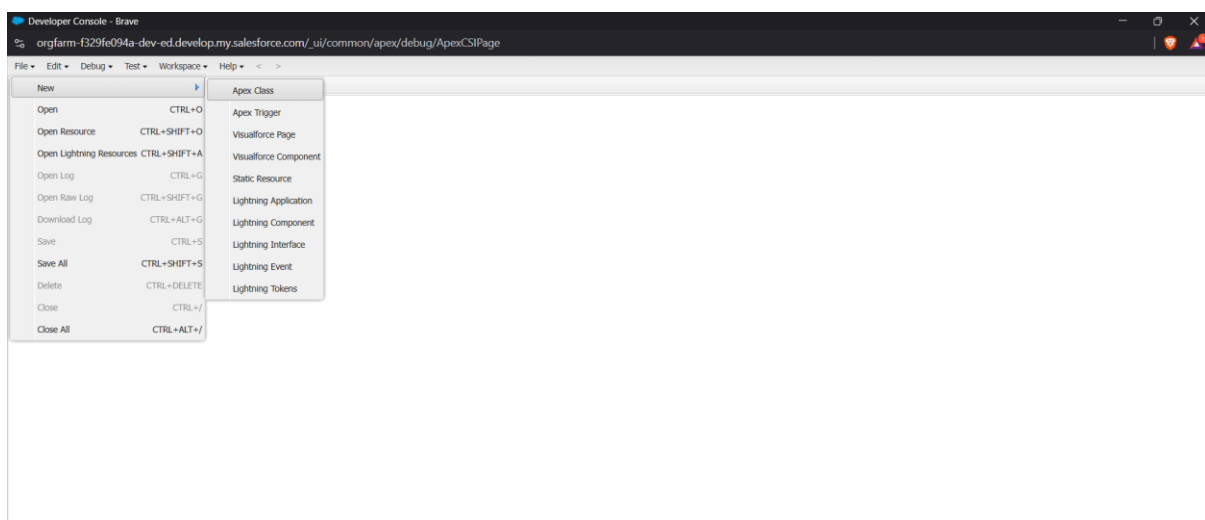
- **Click Gear Icon:** Locate the **Gear Icon** (⚙️) in the top-right corner of the screen.



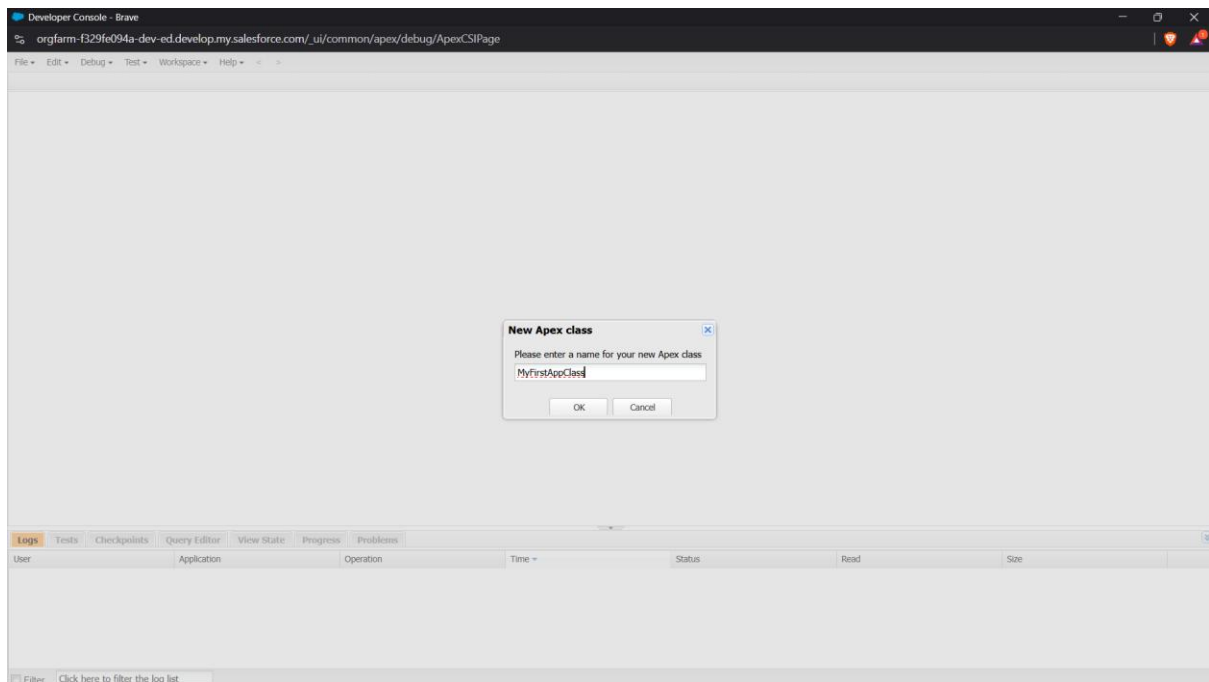
- **Open Developer Console:** From the dropdown menu, select **Developer Console**. This will open a new window which is your Integrated Development Environment (IDE).

3. Creating the Apex Class

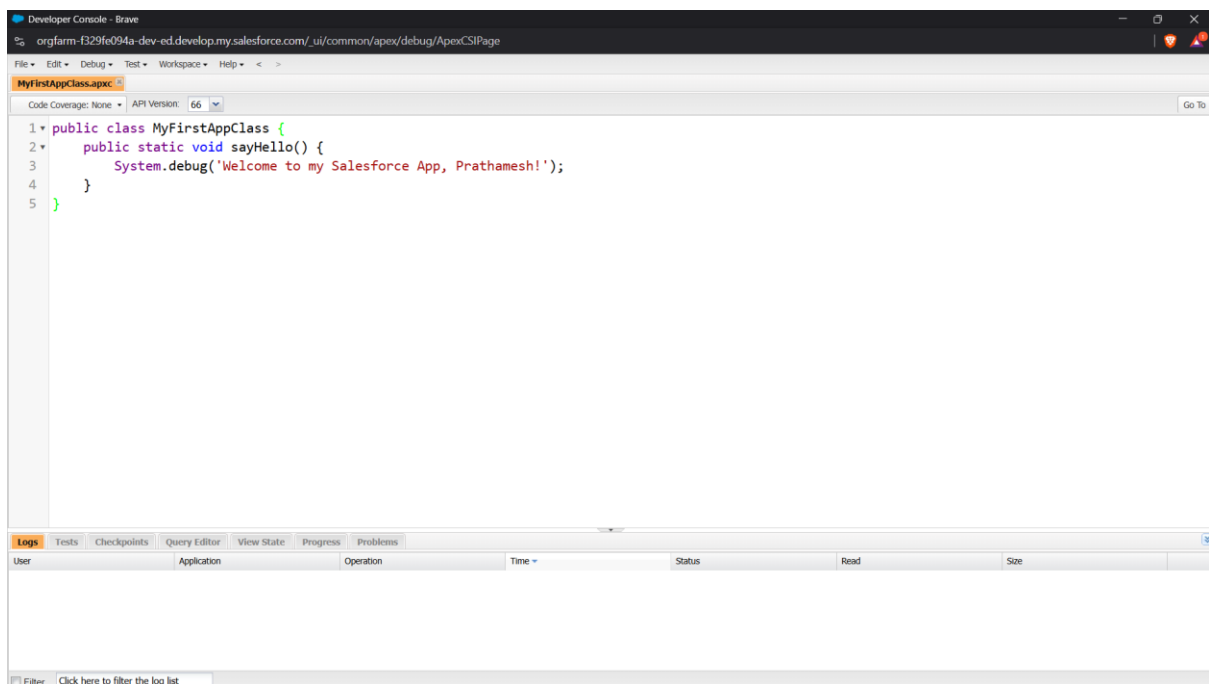
- **Create New File:** In the Developer Console, go to **File > New > Apex Class**.



- **Naming:** Enter the name myFirstAppClass and click **OK**.

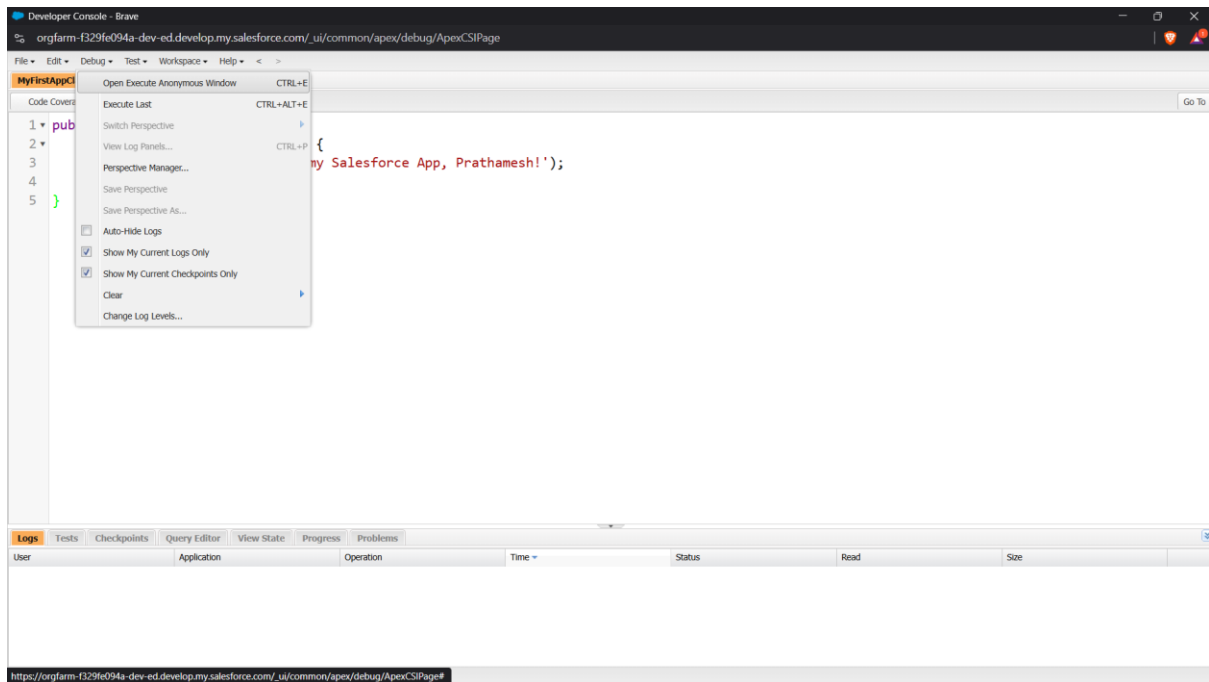


- **Save:** Press Ctrl + S (Windows) or Cmd + S (Mac) to save the file to the cloud.



4. Running the Project (Execution)

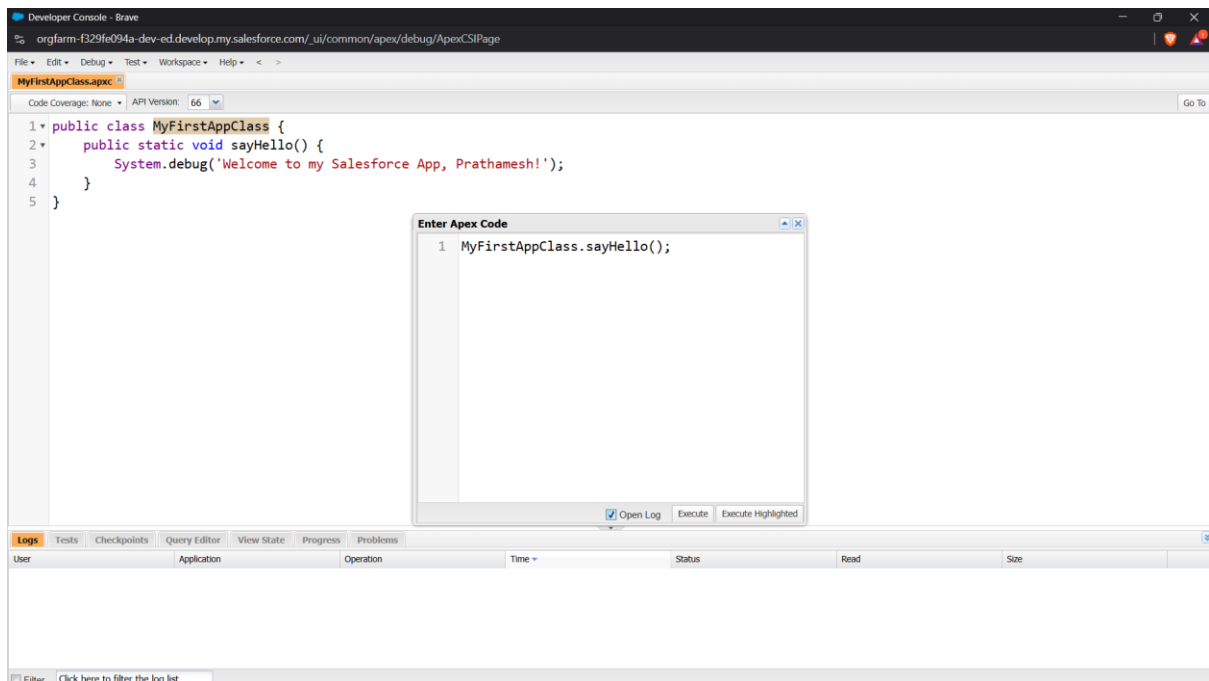
- **Open Debug Window:** Go to the **Debug** menu at the top of the Developer Console.



- **Execute Anonymous:** Click on **Open Execute Anonymous Window**.

Call the Method:

Execute: Click the **Execute** button at the bottom right.



5. Viewing the Output

- **Logs Tab:** A new "Log" entry will appear in the bottom pane. Double-click it.

The screenshot shows the Salesforce Developer Console interface. The top pane displays the Execution Log for the 'Log executeAnonymous @2/24/2026, 7:15:42 PM' entry. The log contains various events such as USER_INFO, EXECUTION_START, CODE_UNIT_START, HEAP_ALLOCATE, STATEMENT_EXECUTE, and METHOD_ENTRY. The bottom pane shows the Logs tab with a table of log entries.

User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Unknown	/services/data/v6.0/tooling/executeA...	2/24/2026, 7:15:42 PM	Success		2.59 KB

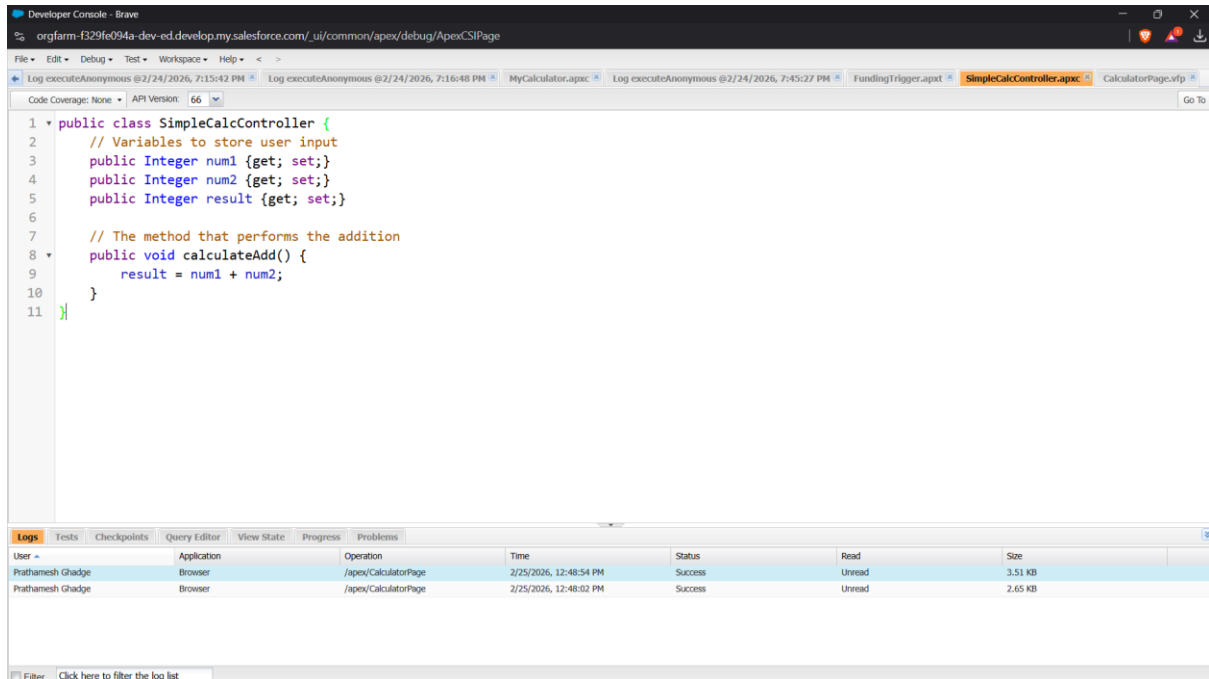
- **Filter Results:** In the new window, check the box that says **Debug Only**.

The screenshot shows the Salesforce Developer Console interface. The top pane displays the Execution Log for the 'Log executeAnonymous @2/24/2026, 7:16:48 PM' entry. The log contains a single event: USER_DEBUG with the message '[3]DEBUG>Welcome to my Salesforce App, Prathamesh!'. The bottom pane shows the Logs tab with a table of log entries. The 'Debug Only' checkbox is checked in the filter section.

User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Unknown	/services/data/v6.0/tooling/executeA...	2/24/2026, 7:16:48 PM	Success		2.57 KB
Prathamesh Ghadge	Unknown	/services/data/v6.0/tooling/executeA...	2/24/2026, 7:15:42 PM	Success		2.59 KB

Part 1: The Logic (Apex Class) :

1. Open **Developer Console**.
2. Go to **File > New > Apex Class**.
3. Name it SimpleCalcController



The screenshot shows the Salesforce Developer Console with the SimpleCalcController Apex class open. The code defines a class with two integer variables (num1, num2) and a result variable, and a calculateAdd() method that returns the sum of num1 and num2.

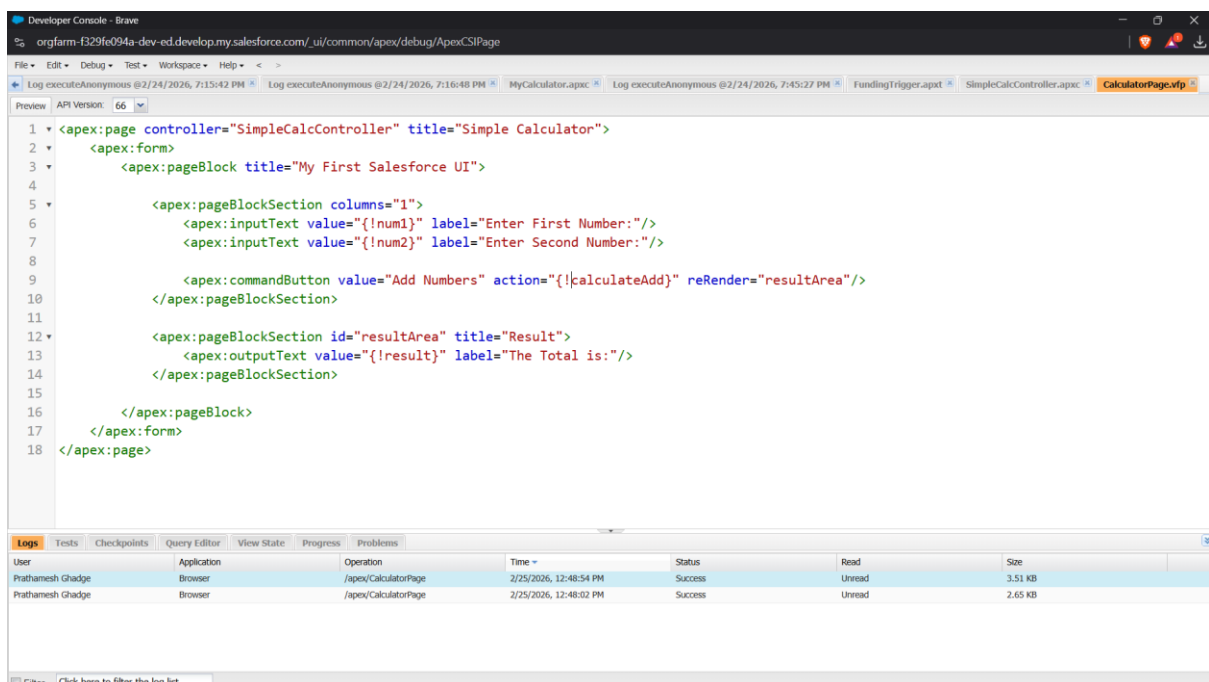
```
1 public class SimpleCalcController {
2     // Variables to store user input
3     public Integer num1 {get; set;}
4     public Integer num2 {get; set;}
5     public Integer result {get; set;}
6
7     // The method that performs the addition
8     public void calculateAdd() {
9         result = num1 + num2;
10    }
11 }
```

The bottom pane shows the Logs tab with a table of recent operations:

User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Browser	/apex/CalculatorPage	2/25/2026, 12:48:54 PM	Success	Unread	3.51 KB
Prathamesh Ghadge	Browser	/apex/CalculatorPage	2/25/2026, 12:48:02 PM	Success	Unread	2.65 KB

Part 2: The UI (Visualforce Page) :

1. In the Developer Console, go to **File > New > Visualforce Page**.
2. Name it CalculatorPage.



The screenshot shows the Salesforce Developer Console with the CalculatorPage Visualforce page open. The code uses Apex components to create a form with two input fields, a button to calculate the sum, and a result area.

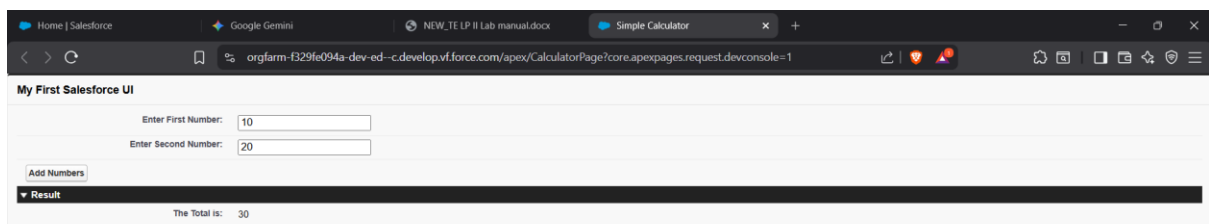
```
1 <apex:page controller="SimpleCalcController" title="Simple Calculator">
2     <apex:form>
3         <apex:pageBlock title="My First Salesforce UI">
4
5             <apex:pageBlockSection columns="1">
6                 <apex:inputText value="{!num1}" label="Enter First Number:"/>
7                 <apex:inputText value="{!num2}" label="Enter Second Number:"/>
8
9                 <apex:commandButton value="Add Numbers" action="{!calculateAdd}" reRender="resultArea"/>
10            </apex:pageBlockSection>
11
12            <apex:pageBlockSection id="resultArea" title="Result">
13                <apex:outputText value="{!result}" label="The Total is:"/>
14            </apex:pageBlockSection>
15
16        </apex:pageBlock>
17    </apex:form>
18 </apex:page>
```

The bottom pane shows the Logs tab with a table of recent operations:

User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Browser	/apex/CalculatorPage	2/25/2026, 12:48:54 PM	Success	Unread	3.51 KB
Prathamesh Ghadge	Browser	/apex/CalculatorPage	2/25/2026, 12:48:02 PM	Success	Unread	2.65 KB

Part 3: How to Run It :

1. On the **Visualforce Page** editor (where you pasted the HTML), look at the top-left toolbar.
2. Click the **Preview** button.
3. A new browser tab will open. Enter two numbers, click the button, and you will see the total appear instantly!

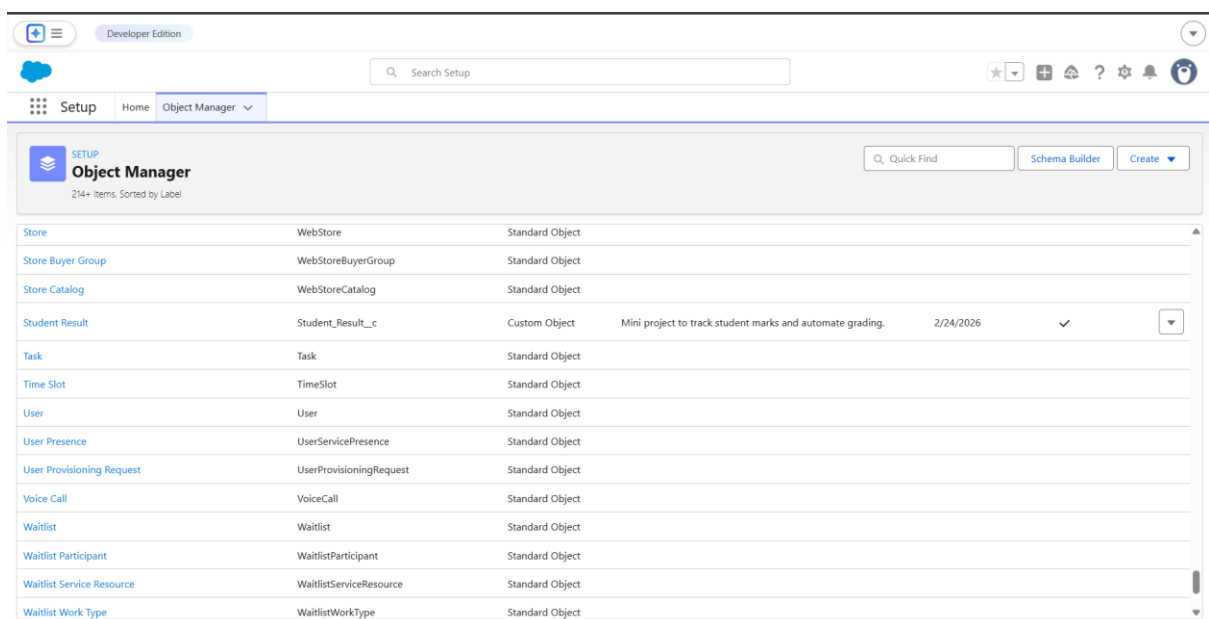


Project Title: Student Result Management System

Objective: To automate the grading process in Salesforce using Apex Triggers and Custom Objects.

Step 1: Create the Custom Object (The Database)

1. Click the **Gear Icon** → **Setup**.
2. Go to **Object Manager** → **Create** → **Custom Object**.
3. Fill in the details:
 - **Label:** Student Result
 - **Plural Label:** Student Results
 - **Record Name:** Student Name (Data Type: **Text**)
4. **Crucial Step:** Under **Optional Features**, check **Allow Reports**.
5. Under **Object Creation Options**, check **Launch New Custom Tab Wizard**.
6. **Save** the object and follow the Tab Wizard (choose any icon) to make it visible in the App Launcher.



Store	WebStore	Standard Object				
Store Buyer Group	WebStoreBuyerGroup	Standard Object				
Store Catalog	WebStoreCatalog	Standard Object				
Student Result	Student_Result__c	Custom Object	Mini project to track student marks and automate grading.	2/24/2026	✓	
Task	Task	Standard Object				
Time Slot	TimeSlot	Standard Object				
User	User	Standard Object				
User Presence	UserServicePresence	Standard Object				
User Provisioning Request	UserProvisioningRequest	Standard Object				
Voice Call	VoiceCall	Standard Object				
Waitlist	Waitlist	Standard Object				
Waitlist Participant	WaitlistParticipant	Standard Object				
Waitlist Service Resource	WaitlistServiceResource	Standard Object				
Waitlist Work Type	WaitlistWorkType	Standard Object				

Step 2: Create Custom Fields

Now, add the "Columns" to your table. In the **Object Manager**, inside your **Student Result** object:

1. Select **Fields & Relationships** → **New**.
2. **Field 1 (Marks):** * Data Type: **Number** | Label: Marks | Length: 3.
3. **Field 2 (Grade):** * Data Type: **Text** | Label: Grade | Length: 5.
4. **Field 3 (Status):** * Data Type: **Text** | Label: Status | Length: 10.

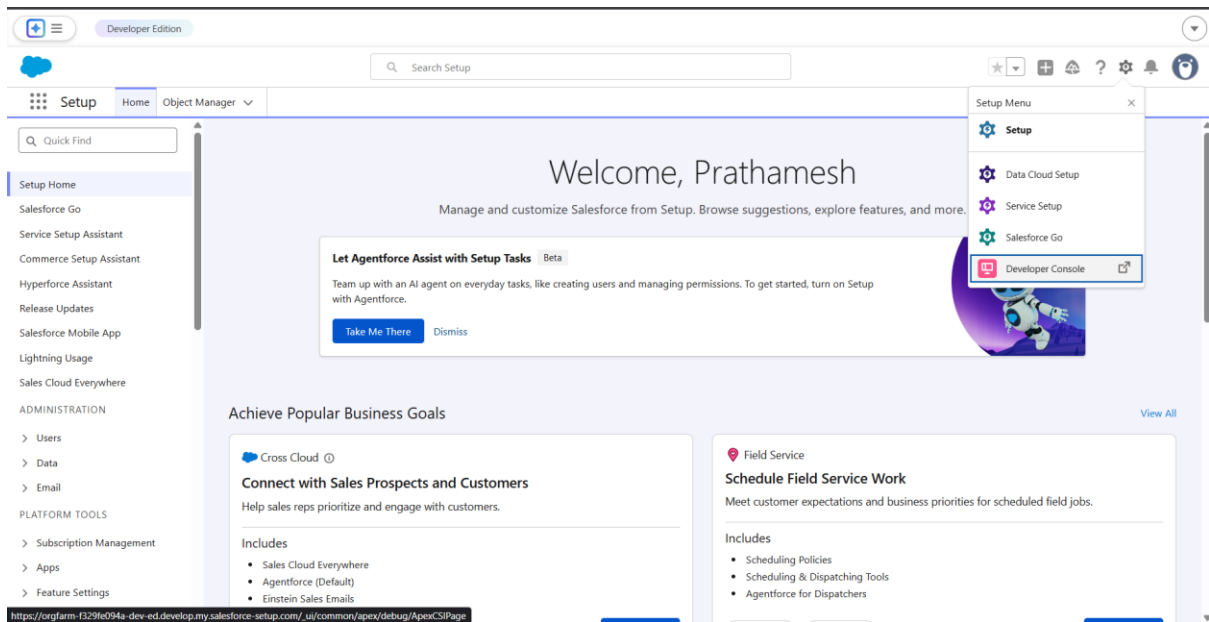
The screenshot shows the Salesforce Object Manager interface for the 'Student Result' object. The 'Fields & Relationships' section is active, displaying a table of 7 fields. The 'Marks' field is highlighted in blue. The table has columns for FIELD LABEL, FIELD NAME, DATA TYPE, CONTROLLING FIELD, and INDEXED.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Grade	Grade__c	Text(10)		
Last Modified By	LastModifiedById	Lookup(User)		
Marks	Marks__c	Number(3, 0)		
Owner	OwnerId	Lookup(User, Group)		✓
Status	Status__c	Text(10)		
Student Result Name	Name	Text(80)		✓

Step 3: Create the Logic (Apex Class)

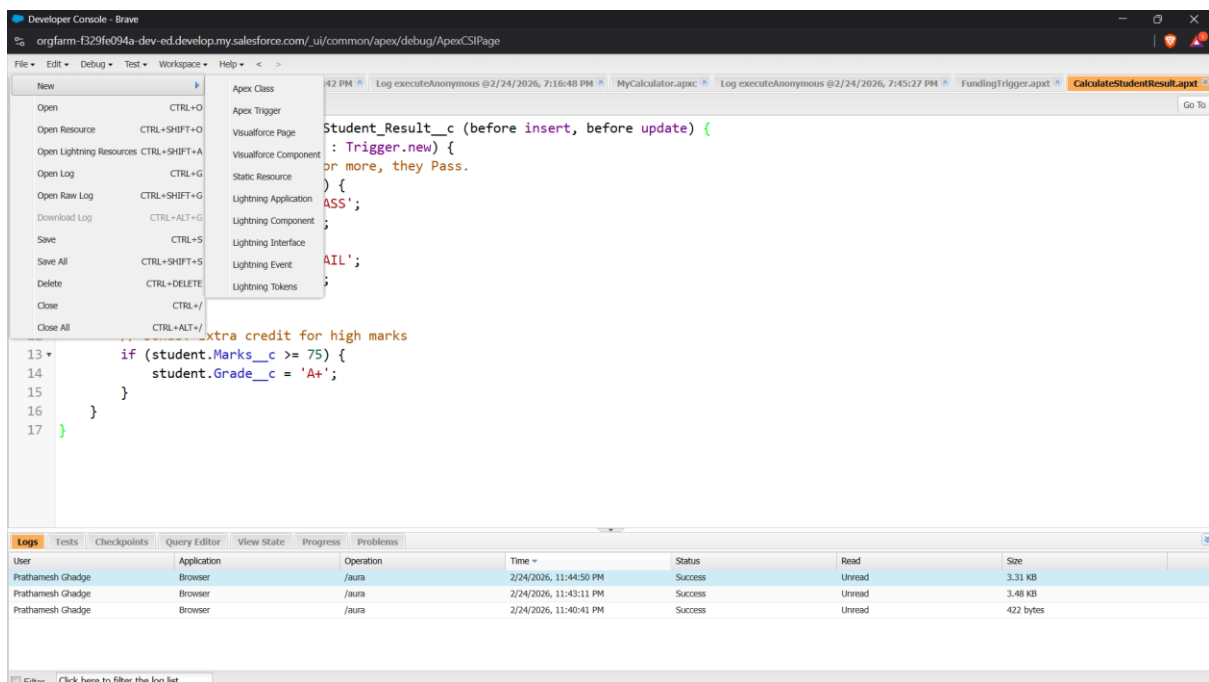
It is "Best Practice" to keep logic in a Class and only call it from a Trigger.

1. Click **Gear Icon** → **Developer Console**.
2. **File > New > Apex Class**. Name: ResultHandler.

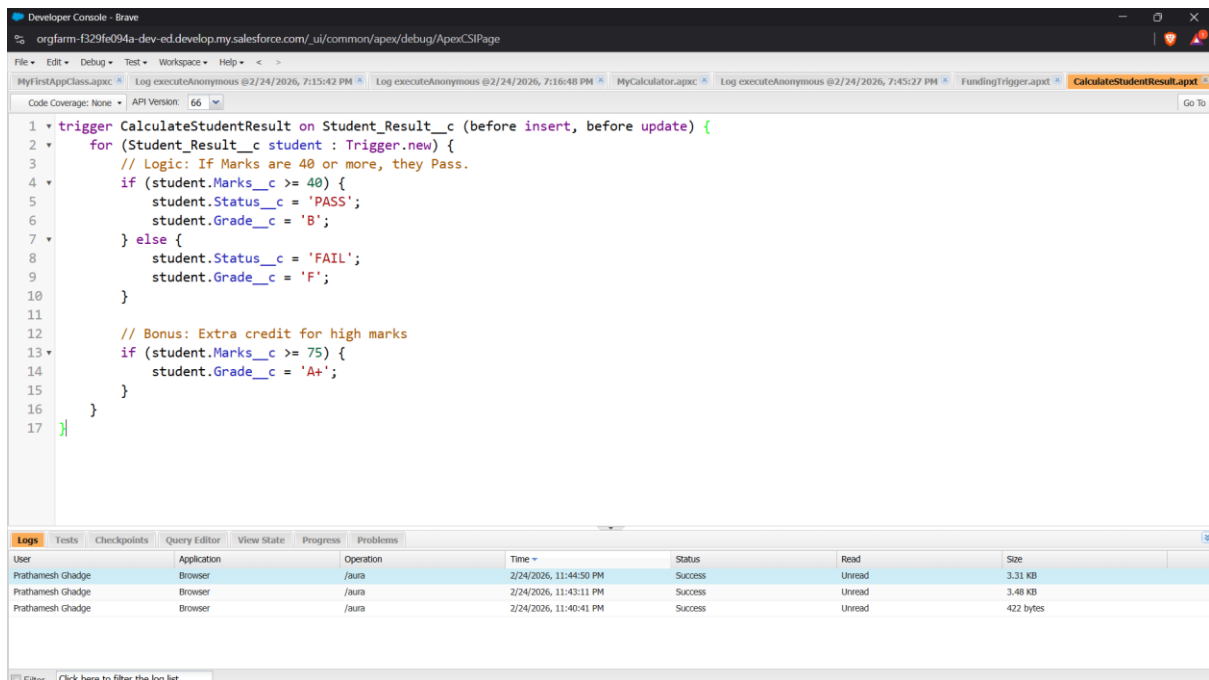


Step 4: Create the Automation (Apex Trigger)

1. In Developer Console: **File > New > Apex Trigger**.
2. **Name:** CalculateStudentResult | **sObject:** Student_Result__c



3. Press **Ctrl + S** to save.

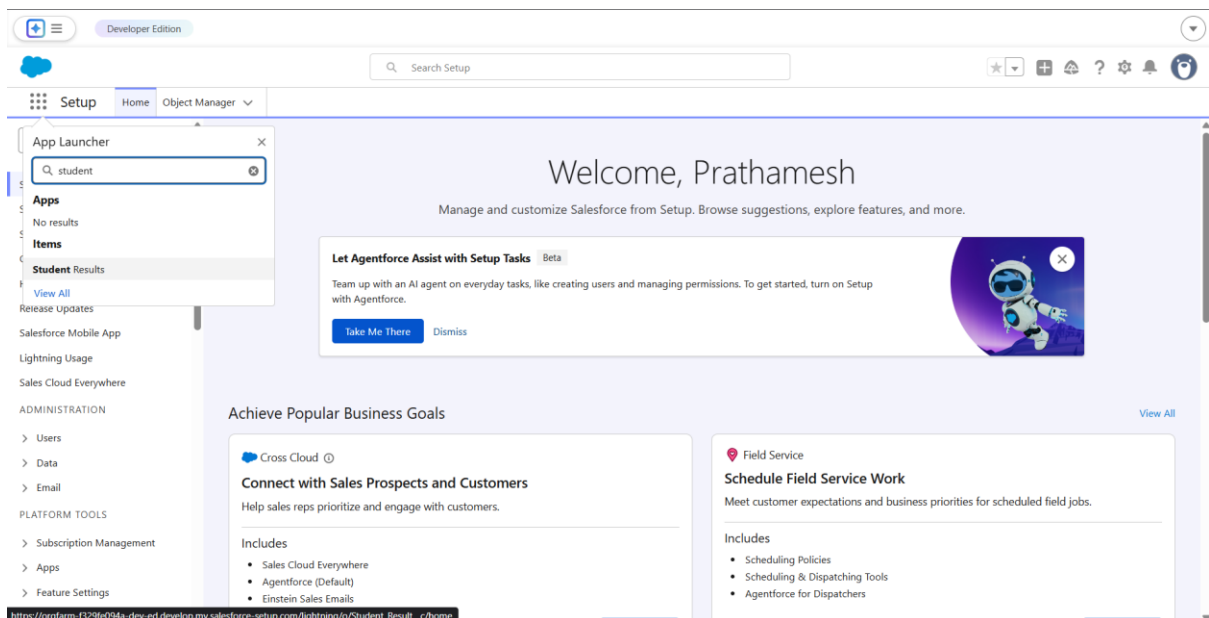


```
1 trigger CalculateStudentResult on Student_Result__c (before insert, before update) {
2   for (Student_Result__c student : Trigger.new) {
3     // Logic: If Marks are 40 or more, they Pass.
4     if (student.Marks__c >= 40) {
5       student.Status__c = 'PASS';
6       student.Grade__c = 'B';
7     } else {
8       student.Status__c = 'FAIL';
9       student.Grade__c = 'F';
10    }
11  }
12  // Bonus: Extra credit for high marks
13  if (student.Marks__c >= 75) {
14    student.Grade__c = 'A+';
15  }
16 }
17 }
```

User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Browser	/aura	2/24/2026, 11:44:50 PM	Success	Unread	3.31 KB
Prathamesh Ghadge	Browser	/aura	2/24/2026, 11:43:11 PM	Success	Unread	3.48 KB
Prathamesh Ghadge	Browser	/aura	2/24/2026, 11:40:41 PM	Success	Unread	422 bytes

Step 5: Execution & Output (Testing)

1. Go to your browser tab → **App Launcher** (9 dots).
2. Search and click **Student Results**.



1. Click **New**.
2. **Input:** Student Name: Heena
 - Marks: 95

- (Leave Grade and Status empty)

3. Click **Save**.

The screenshot shows a 'New Student Result' modal form. The form has a title bar with a close button. Below the title bar, there's a section labeled 'Information' with a red asterisk indicating required information. The form contains the following fields: 'Student Result Name' (filled with 'Heena'), 'Marks' (filled with '95'), 'Grade' (empty), and 'Status' (empty). To the right of these fields, the 'Owner' is listed as 'Prathamesh Ghadge'. At the bottom of the modal, there are three buttons: 'Cancel', 'Save & New', and 'Save'.

Step 6: Expected Output for Documentation

When you take screenshots for your report, show these three things:

1. **The Record Detail Page:** Show that **Grade** and **Status** were filled automatically.

The screenshot shows the 'Student Result' record detail page for 'Heena'. At the top, there's a green notification banner that says 'Student Result "Heena" was created.' The page has a header with 'Student Result' and 'Heena'. Below the header, there's a 'Details' tab. The details section shows the following information: 'Student Result Name' (Heena), 'Marks' (95), 'Grade' (A+), 'Status' (PASS), 'Created By' (Prathamesh Ghadge, 2/24/2026, 10:21 AM), and 'Last Modified By' (Prathamesh Ghadge, 2/24/2026, 10:21 AM). To the right of the details, there's an 'Activity' section with a 'New Contact', 'Edit', and 'New Opportunity' button. The activity section shows a list of activities with filters for 'All time', 'All activities', and 'All types'. The activity list is currently empty, showing 'No activities to show. Get started by sending an email, scheduling a task, and more.' At the bottom of the activity section, there's a 'Show All Activities' button.

Developer Edition

Search...

★

+

🏠

?

⚙️

🔔

👤

Developer Edition

Welcome

Student Results

Recently Viewed

New

Import

Change Owner

Assign Label

3 items • Updated a few seconds ago

Search this list...

⚙️

📄

🔄

🔍

✏️

🗑️

🔼

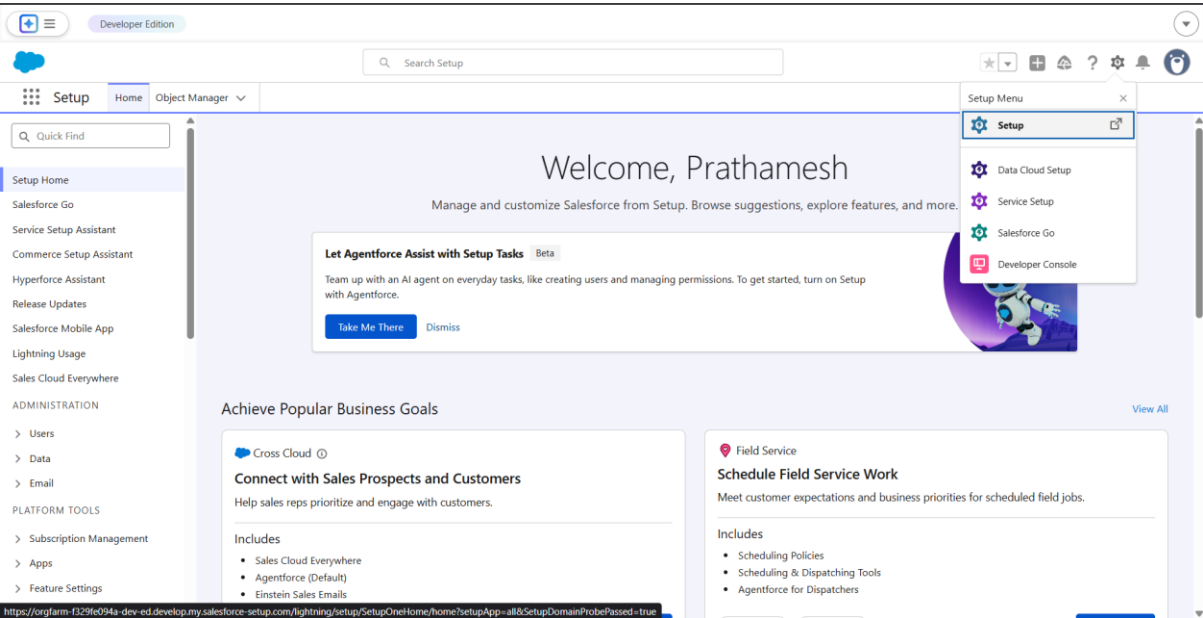
	☐ Student Result Name	
1	☐ Heena	🔼
2	☐ Radhey	🔼
3	☐ Prathamesh	✏️ 🔼

Project Title: AI-Powered Talent Sourcing & Recruitment Tracker

Phase 1: Database Setup (The "Object")

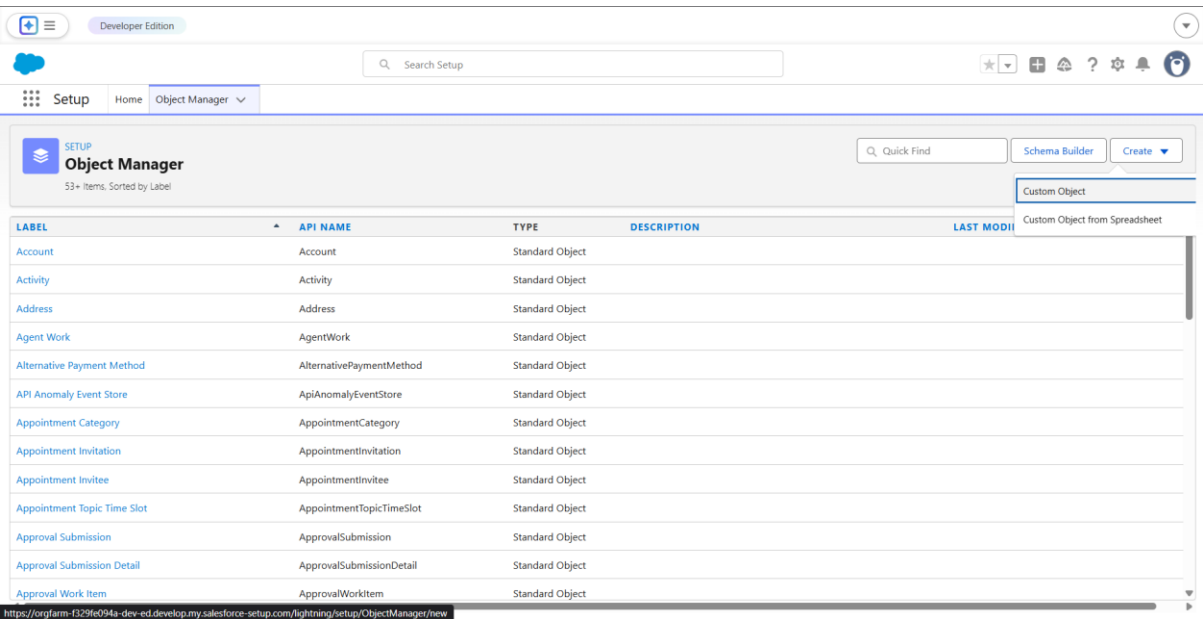
We need a place to store candidate data.

1. Login to Salesforce → Gear Icon → Setup.



2. Go to Object Manager → Create → Custom Object.

- **Label:** Applicant
- **Plural Label:** Applicants
- **Record Name:** Applicant Name (Data Type: Text)



SETUP
New Custom Object

New Custom Object

Help for this Page

Permissions for this object are disabled for all profiles by default. You can enable object permissions in permission sets or by editing custom profiles. [Tell me more!](#) [Don't show this message again](#)

Custom Object Definition Edit
Save Save & New Cancel

Custom Object Information
Required Information

The singular and plural labels are used in tabs, page layouts, and reports.

Label Example: Account

Plural Label Example: Accounts

Starts with vowel sound ☐

The Object Name is used when referencing the object via the API.

Object Name Example: Account

Description

Context-Sensitive Help Setting

☒ Open the standard Salesforce.com Help & Training window

☐ Open a window using a Visualforce page

Content Name

3. **Check the box: "Allow Reports" and "Launch New Custom Tab Wizard". Save.**

SETUP
New Custom Object

Developer Edition

Search Setup

Setup Home Object Manager

Optional Features

- ☒ Allow Reports
- ☒ Allow Activities
- ☒ Track Field History
- ☐ Allow in Chatter Groups
- ☐ Enable Licensing

Object Classification

When these settings are enabled, this object is classified as an Enterprise Application object. When these settings are disabled, this object is classified as a Light Application object. [Learn more.](#)

- ☒ Allow Sharing
- ☒ Allow Bulk API Access
- ☒ Allow Streaming API Access

Deployment Status

[What is this?](#)

☐ In Development

☒ Deployed

Search Status

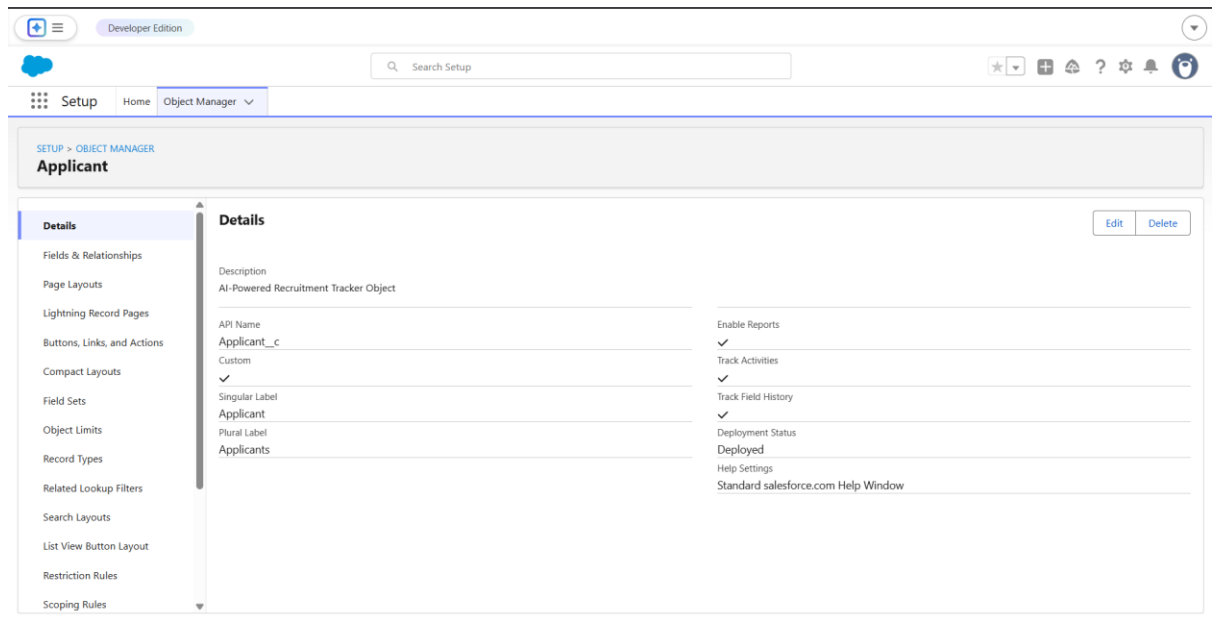
When this setting is enabled, your users can find records of this object type when they search. [Learn more.](#)

☒ Allow Search

Object Creation Options (Available only when custom object is first created)

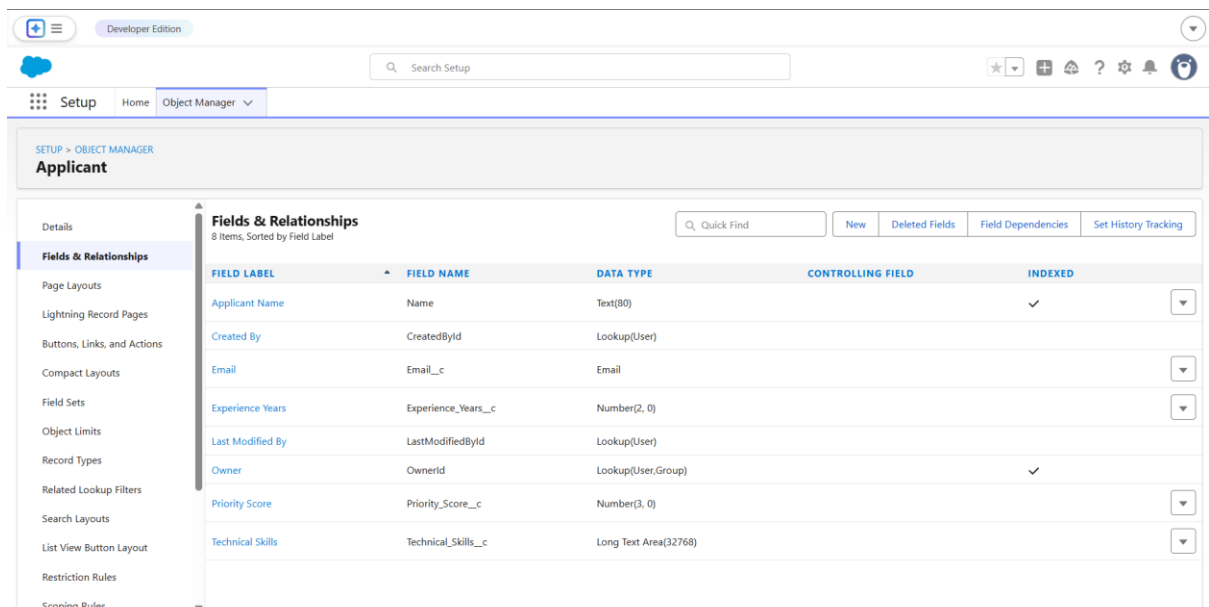
- ☒ Add Notes and Attachments related list to default page layout
- ☒ Launch New Custom Tab Wizard after saving this custom object

4. Follow the Tab Wizard to pick an icon and click **Save**.



5. **Add Custom Fields:** Inside the **Applicant** object, go to **Fields & Relationships** and click **New** for each of these:

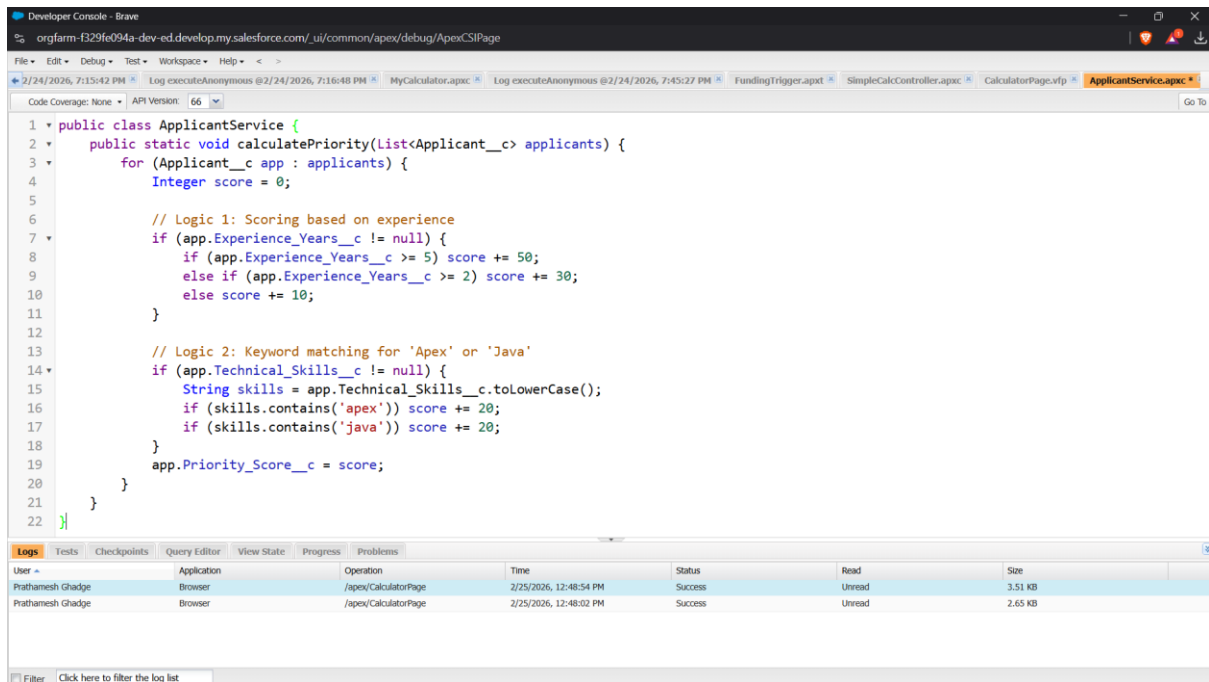
- **Email:** (Data Type: **Email**) | Label: Email
- **Experience:** (Data Type: **Number**, 2, 0) | Label: Experience Years
- **Technical Skills:** (Data Type: **Long Text Area**) | Label: Technical Skills
- **Priority Score:** (Data Type: **Number**, 3, 0) | Label: Priority Score



Phase 2: Backend Logic (The "Brain")

We will create a service class to handle the "AI" scoring.

1. Click **Gear Icon** $\rightarrow$ **Developer Console**.
2. **File > New > Apex Class**. Name: ApplicantService.
3. Save (Ctrl+S).



The screenshot shows the Salesforce Developer Console with the 'ApplicantService.apex' class open. The class contains two logic blocks: one for scoring based on experience and another for keyword matching on technical skills. Below the code editor, the 'Logs' tab is active, displaying a table of recent log entries.

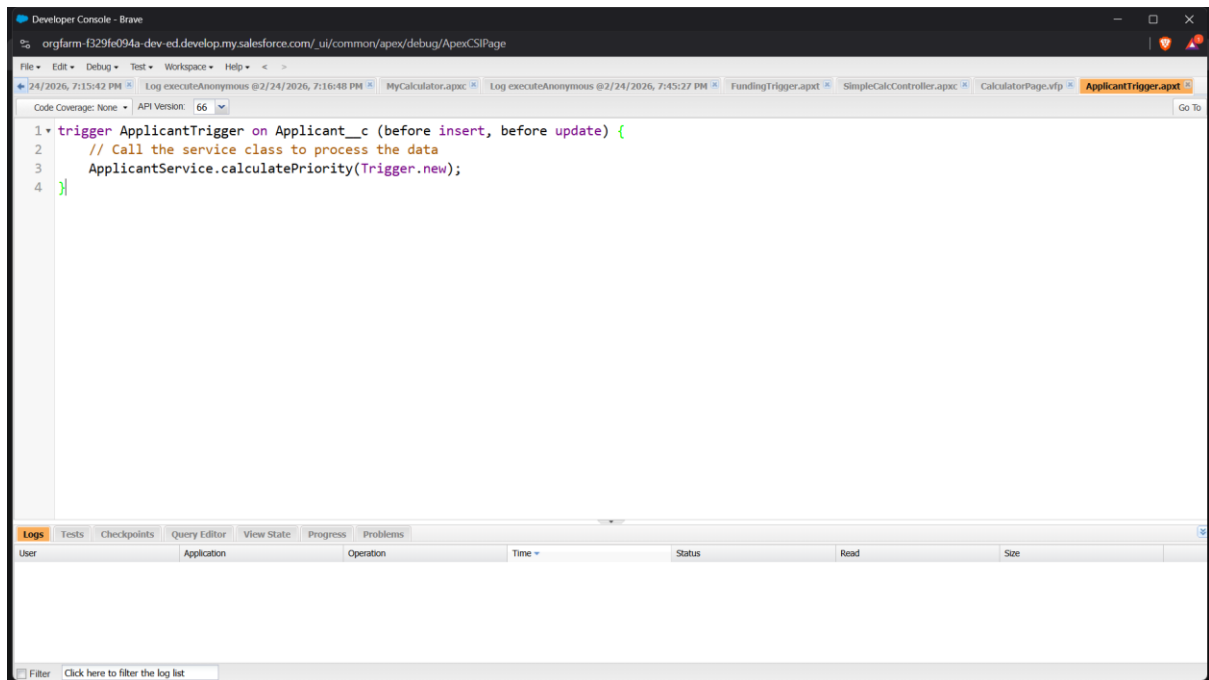
```
1 public class ApplicantService {
2     public static void calculatePriority(List<Applicant__c> applicants) {
3         for (Applicant__c app : applicants) {
4             Integer score = 0;
5
6             // Logic 1: Scoring based on experience
7             if (app.Experience_Years__c != null) {
8                 if (app.Experience_Years__c >= 5) score += 50;
9                 else if (app.Experience_Years__c >= 2) score += 30;
10                else score += 10;
11            }
12
13            // Logic 2: Keyword matching for 'Apex' or 'Java'
14            if (app.Technical_Skills__c != null) {
15                String skills = app.Technical_Skills__c.toLowerCase();
16                if (skills.contains('apex')) score += 20;
17                if (skills.contains('java')) score += 20;
18            }
19            app.Priority_Score__c = score;
20        }
21    }
22 }
```

User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Browser	/apex/CalculatorPage	2/25/2026, 12:48:54 PM	Success	Unread	3.51 KB
Prathamesh Ghadge	Browser	/apex/CalculatorPage	2/25/2026, 12:48:02 PM	Success	Unread	2.65 KB

Phase 3: Automation (The "Trigger")

This ensures the score is calculated the second you click "Save" in the UI.

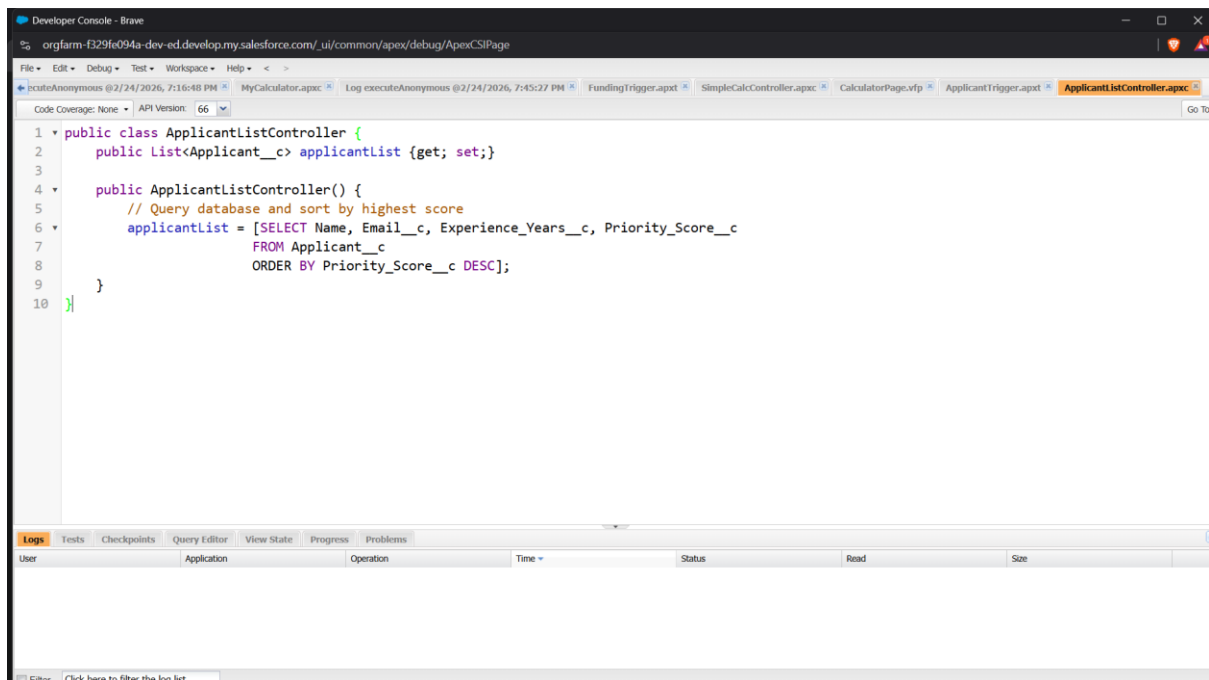
1. In Developer Console: **File > New > Apex Trigger**.
2. **Name:** ApplicantTrigger | **sObject:** Applicant__c.
3. Save (Ctrl+S).



Phase 4: Custom User Interface (The "Leaderboard")

This is where we show the results in a professional table view.

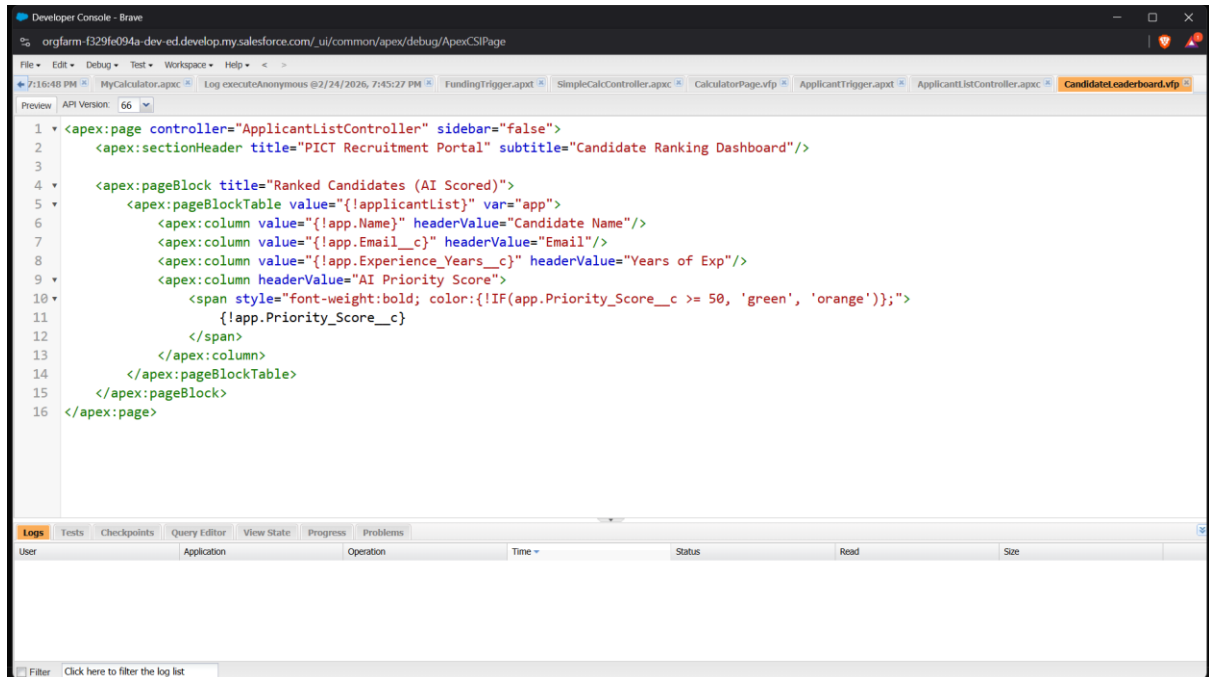
1. In Developer Console: **File > New > Apex Class**. Name: ApplicantListController.
2. Save (Ctrl+S)



Phase 5: Create Visualforce Page

1. **File > New > Visualforce Page.** Name: CandidateLeaderboard.

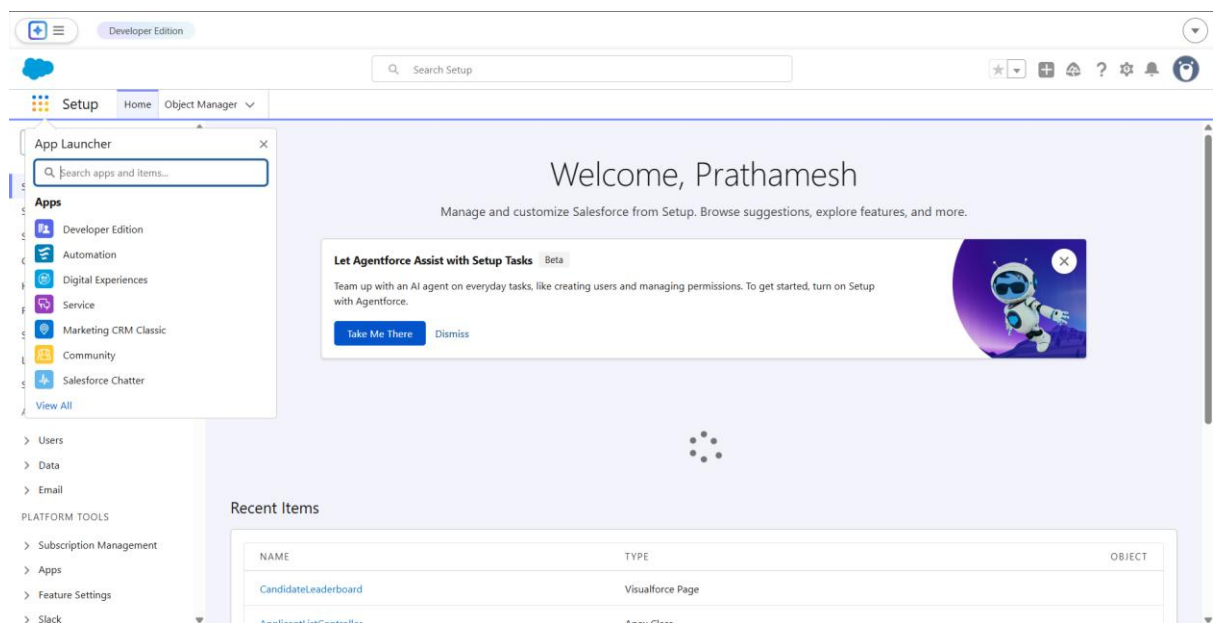
2. Save (Ctrl+S)

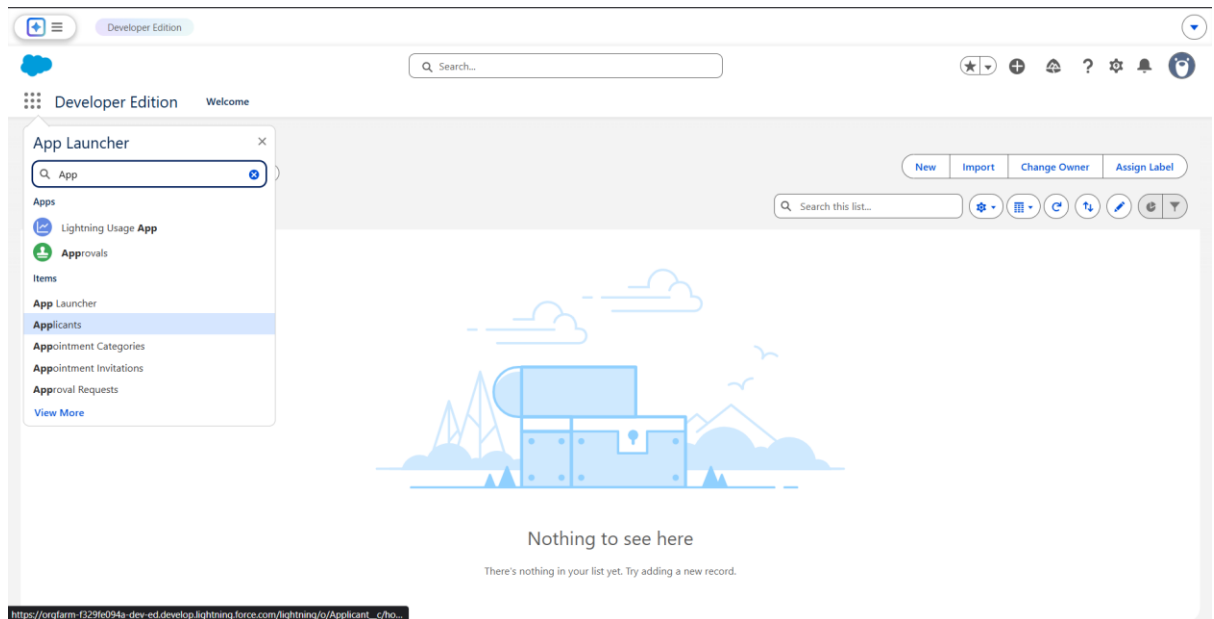


```
1 <apex:page controller="ApplicantListController" sidebar="false">
2   <apex:sectionHeader title="PICT Recruitment Portal" subtitle="Candidate Ranking Dashboard"/>
3
4   <apex:pageBlock title="Ranked Candidates (AI Scored)">
5     <apex:pageBlockTable value="{!applicantList}" var="app">
6       <apex:column value="{!app.Name}" headerValue="Candidate Name"/>
7       <apex:column value="{!app.Email__c}" headerValue="Email"/>
8       <apex:column value="{!app.Experience_Years__c}" headerValue="Years of Exp"/>
9       <apex:column headerValue="AI Priority Score">
10        <span style="font-weight:bold; color:{!IF(app.Priority_Score__c >= 50, 'green', 'orange')};">
11          {!app.Priority_Score__c}
12        </span>
13      </apex:column>
14    </apex:pageBlockTable>
15  </apex:pageBlock>
16</apex:page>
```

Phase 6: Implementation & Testing

1. **Input Data:** Go to the **App Launcher** (9 dots) → **Applicants** → Click **New**.





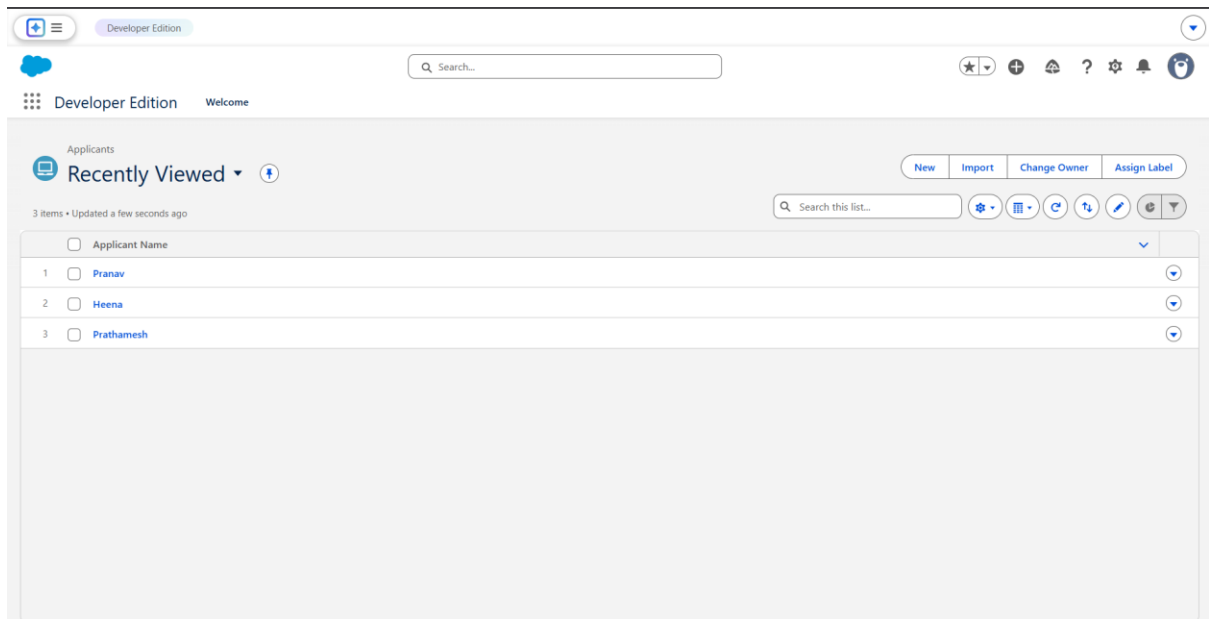
2. Create 3 records:

- *Candidate A*: 6 years Exp, Skills: "Apex, Java" (Expected Score: 90)
- *Candidate B*: 3 years Exp, Skills: "Python" (Expected Score: 30)
- *Candidate C*: 1 year Exp, Skills: "Java" (Expected Score: 30)

The screenshot shows the 'New Applicant' form in the Salesforce Developer Edition. The form is titled 'New Applicant' and contains the following fields:

- Applicant Name**: Prathamesh
- Email**: prathamtech24@gmail.com
- Experience Years**: 6
- Technical Skills**: Apex, Java
- Priority Score**: (empty)

The Owner is set to Prathamesh Ghadge. The form also includes a 'Cancel' button, a 'Save & New' button, and a 'Save' button. The background shows the 'Applicants' list with 'Recently Viewed' and '0 items • Updated 3 minutes ago'.



3. **View Results:** Go back to your **Visualforce Page** in the Developer Console and click **Preview**.

PICT Recruitment Portal
Candidate Ranking Dashboard

Ranked Candidates (AI Scored)			
Candidate Name	Email	Years of Exp	AI Priority Score
Prathamesh	prathamtech24@gmail.com	6	90
Heena	heena14@gmail.com	2	80
Pranav	pranav29@gmail.com	3	30

